

Lecture-#2

Natural Language Processing

A8706

Introduction

Text Tokenization

Corpus:

A Corpus is defined as a collection of text documents.

For Example,

A data set containing news is a corpus or
The tweets containing Twitter data are a corpus.

Text Tokenization

Tokens:

Tokens can be words, phrases, or Engrams, and Engrams are defined as the group of n words together.

For example, consider the sentence given below:

```
Sentence: I like my iphone
```

For the above sentence, the different engrams are as follows:

```
Uni-grams( $n=1$ ) are: I, like, my, iphone
```

```
Di-grams( $n=2$ ) are: I like, like my, my iphone
```

```
Tri-grams( $n=3$ ) are: I like my, like my iphone
```

Corpus > Documents > Paragraphs > Sentences > Tokens

Text Tokenization

ngrams:

```
!pip install nltk
```

```
from nltk import ngrams
```

```
sentence = 'Hello everyone. Welcome to class. You are studying Web and Text Analytics article'
```

```
n = 2
```

```
x = ngrams(sentence.split(), n)
```

```
for grams in x:
```

```
    print (grams)
```

O/P:

```
('Hello', 'everyone.') ('everyone.', 'Welcome') ('Welcome', 'to') ('to', 'class.') ('class.', 'You') ('You', 'are') ('are', 'studying') ('studying', 'Web') ('Web', 'and') ('and', 'Text') ('Text', 'Analytics') ('Analytics', 'article')
```

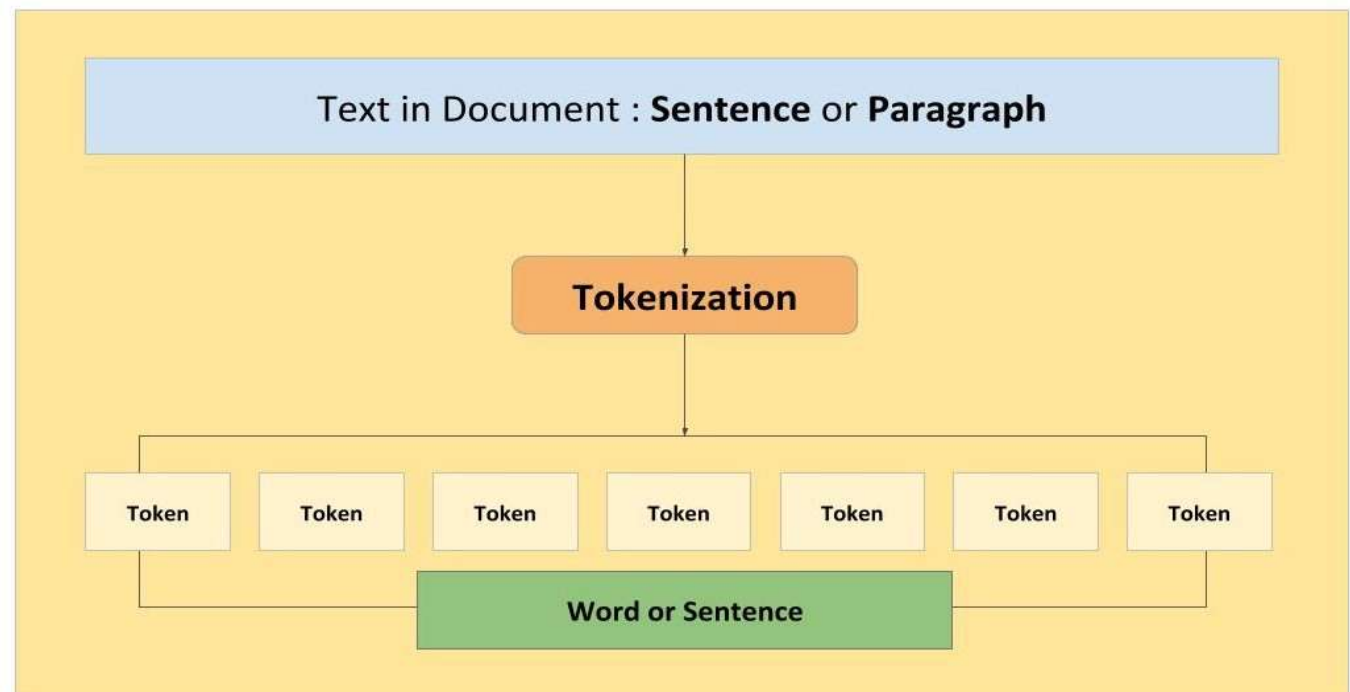
Text Tokenization

Tokenization:

It is the process of converting a text into tokens.

Types:

- ❖ Sentence Tokenization
- ❖ Word Tokenization



Sentence Tokenization

Sentence tokenization is the process of splitting a text corpus into sentences that act as the first level of tokens which the corpus is comprised of. This is also known as *sentence segmentation*, because we try to segment the text into meaningful sentences. Any text corpus is a body of text where each paragraph comprises several sentences.

Sentences are separated by a full stop, so the process of sentence tokenization finds all the full stops in a piece of text to split the data into sentences.

```
!pip install nltk  
import nltk
```

Sentence Tokenization

- ❖ **PUNKT** is an unsupervised trainable model, which means it can be trained on unlabeled data (Data that has not been tagged with information identifying its characteristics, properties, or categories is referred to as unlabeled data.)
- ❖ It contains a variety of libraries for various purposes like text classification, parsing, stemming, tokenizing, etc.
- ❖ **punkt** is the required package for tokenization. Hence you may download it using nltk download manager or download it programmatically using
`nltk.download('punkt').`

Sentence Tokenization

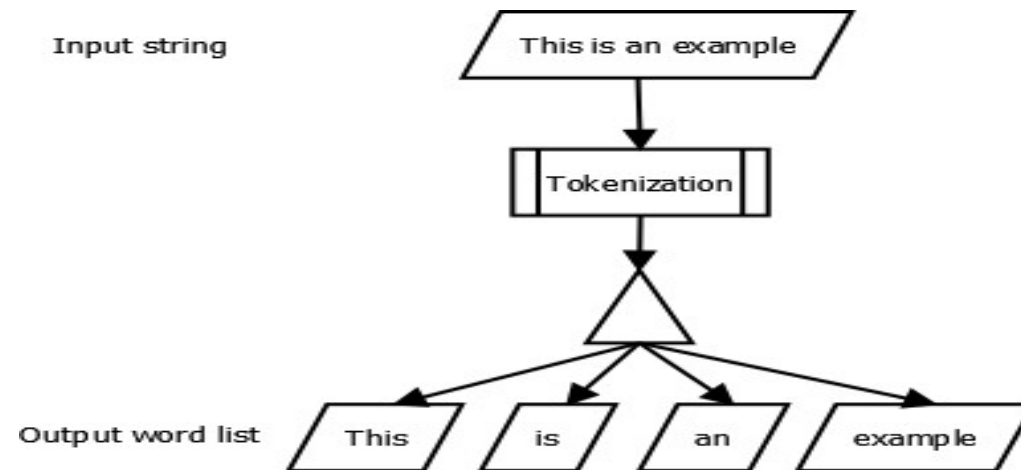
```
!pip install nltk
import nltk
nltk.download('punkt')
from nltk.tokenize import sent_tokenize
text = "Hello everyone. Welcome to class. You are studying Web and Text Analytics article"
sent_tokenize(text)
```

OUTPUT:

```
['Hello everyone.', 'Welcome to class.', 'You are studying Web and Text Analytics article']
```


Word Tokenization

- ❖ Word tokenization is the process of splitting a large sample of text into words.
- ❖ This is a requirement in natural language processing tasks where each word needs to be captured and subjected to further analysis like classifying and counting them for a particular sentiment etc.



Word Tokenization

```
from nltk.tokenize import word_tokenize
```

```
text = "Hello everyone. Welcome to Web and Text Analytic class"  
word_tokenize(text)
```

O/P:

```
['Hello', 'everyone', '.', 'Welcome', 'to', 'Web', 'and', 'Text', 'Analytic', 'class']
```

Word Tokenization

Regular expression (RE): It is a set of characters or a pattern that is used to find substrings in a given string.

Example:

Language

$L = \{\epsilon, 00, 0000, 000000, \dots\}$

RE

$R = (00)^*$

Language

$\{\text{google, googoogle, googoogoogle, googooogoogle, ...}\}$

RE

$R = g(oog)^+le$

Word Tokenization

RegexpTokenizer:

```
from nltk.tokenize import RegexpTokenizer
tokenizer2 = RegexpTokenizer(r"[\w]+")
text = "Hello everyone. Welcome to Web and Text Analytic class"
print(tokenizer2.tokenize(text))
```

O/P:

```
['Hello', 'everyone', 'Welcome', 'to', 'Web', 'and', 'Text', 'Analytic',  
'class']
```

Word Tokenization

RegexTokenizer:

Special Sequences	
<code>\b</code>	Word boundary (zero width)
<code>\d</code>	Any decimal digit (equivalent to <code>[0-9]</code>)
<code>\D</code>	Any non-digit character (equivalent to <code>[^0-9]</code>)
<code>\s</code>	Any whitespace character (equivalent to <code>[\t\n\r\f\v]</code>)
<code>\S</code>	Any non-whitespace character (equivalent to <code>[^ \t\n\r\f\v]</code>)
<code>\w</code>	Any alphanumeric character (equivalent to <code>[a-zA-Z0-9_]</code>)
<code>\W</code>	Any non-alphanumeric character (equivalent to <code>[^a-zA-Z0-9_]</code>)

Word Tokenization

Punctuation:

```
# import string library function  
import string
```

```
# Storing the sets of punctuation in variable result  
result = string.punctuation
```

```
# Printing the punctuation values  
print(result)
```

O/P:

```
!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

Word Tokenization

WordPunctTokenizer:

WordPunctTokenizer is used for separating the punctuation from the words.

```
from nltk.tokenize import WordPunctTokenizer
tokenizer1 = WordPunctTokenizer()
tokenizer1.tokenize(text)
```

O/P:

```
['Hello', 'everyone', '.', 'Welcome', 'to', 'class', '.', 'You', 'are', 'studying',  
'Web', 'and', 'Text', 'Analytics', 'article']
```

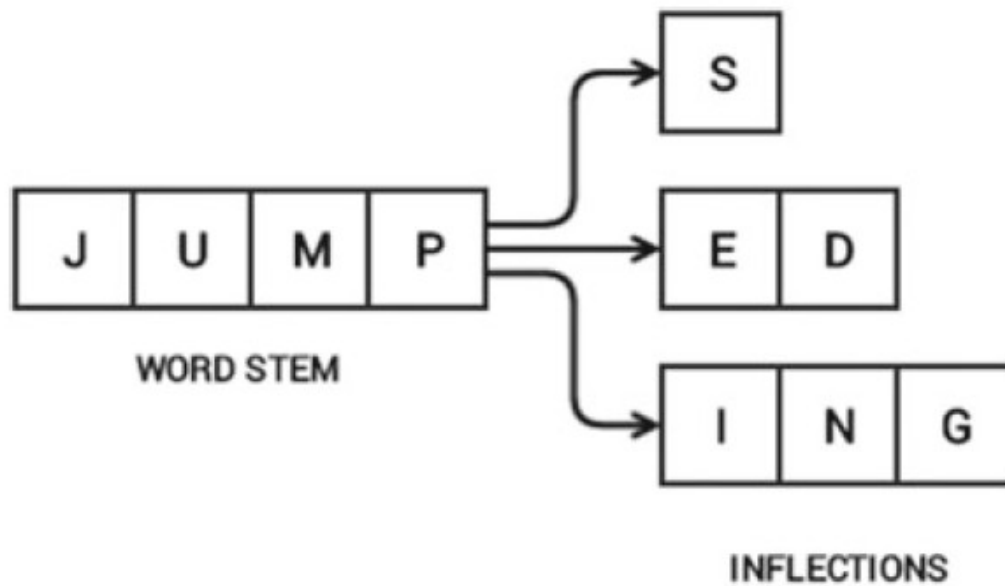
Stemming

Stemming:

- ❖ **Stemming** is the process of producing morphological variants of a root/base word.
- ❖ Stemming programs are commonly referred to as stemming algorithms or stemmers.
- ❖ A stemming algorithm reduces the words “chocolates”, “chocolatey”, and “choco” to the root word, “chocolate” and “retrieval”, “retrieved”, “retrieves” reduce to the stem “retrieve”.
- ❖ **Errors in Stemming:** There are mainly two errors in stemming –
Overstemming and ***Understemming***.
- ❖ Overstemming occurs when two words are stemmed from the same root that are of different stems.
- ❖ Under-stemming occurs when two words are stemmed from the same root that is not of different stems.

Stemming

Stemming:



Stemming

Stemming:

Example: *university* and *universe*

Overstemming: *univers*

data” and “datum

Understemming: *dat* and *datu*

Applications of stemming are:

- ❖ Stemming is used in information retrieval systems like search engines.
- ❖ It is used to determine domain vocabularies in domain analysis.
- ❖ Stemming is desirable as it may reduce redundancy as most of the time the word stem and their inflected/derived words mean the same.

Types of Stemmers in NLTK

Porter's Stemmer

- ❖ It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes.

Example: EED -> EE means “if the word has at least one vowel and consonant plus EED ending, change the ending to EE” as ‘agreed’ becomes ‘agree’.

- **Advantage:** It produces the best output as compared to other stemmers and it has less error rate.
- **Limitation:** Morphological variants produced are not always real words.

Stemming

```
from nltk.stem import PorterStemmer

# Create a Porter Stemmer instance
porter_stemmer = PorterStemmer()

# Example words for stemming
words = ["running", "jumps", "happily", "running", "happily"]

# Apply stemming to each word
stemmed_words = [porter_stemmer.stem(word) for word in words]

# Print the results
print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

Types of Stemmers in NLTK

Lovins Stemmer

- ❖ removes the longest suffix from a word then the word is recorded to convert this stem into valid words.

Example: sitting -> sitt -> sit

- **Advantage:** It is fast and handles irregular plurals like 'teeth' and 'tooth' etc.
- **Limitation:** It is time consuming and frequently fails to form words from stem.

Stemming

```
pip install stemming  
from stemming.lovins import stem
```

```
word = "jumping"  
stemmed_word = stem(word)
```

```
print(stemmed_word)
```

Types of Stemmers in NLTK

Dawsons Stemmer

- ❖ It is an extension of Lovins stemmer in which suffixes are stored in the reversed order indexed by their length and last letter. Example: sitting -> sitt -> sit
- **Advantage:** It is fast in execution and covers more suffices.
- **Limitation:** It is very complex to implement.

Types of Stemmers in NLTK

Krovetz Stemmer

- 1) Convert the plural form of a word to its singular form.
- 2) Convert the past tense of a word to its present tense and remove the suffix 'ing'.

Example: 'children' -> 'child'

Advantage: It is light in nature and can be used as pre-stemmer for other stemmers.

Limitation: It is inefficient in case of large documents.

Types of Stemmers in NLTK

Xerox Stemmer

Capable of processing extensive datasets and generating valid words, it has a tendency to over-stem, primarily due to its reliance on lexicons, making it language-dependent. This constraint implies that its effectiveness is limited to specific languages.

Example:

'children' -> 'child'

'understood' -> 'understand'

'whom' -> 'who'

'best' -> 'good'

Types of Stemmers in NLTK

N-Gram Stemmer

The algorithm, aptly named n-grams (typically $n=2$ or 3), involves breaking words into segments of length n and then applying statistical analysis to identify patterns. An n-gram is a set of n consecutive characters extracted from a word in which similar words will have a high proportion of n-grams in common.

Example: 'INTRODUCTIONS' for $n=2$ becomes : *I, IN, NT, TR, RO, OD, DU, UC, CT, TI, IO, ON, NS, S*

- **Advantage:** It is based on string comparisons and it is language dependent.
- **Limitation:** It requires space to create and index the n-grams and it is not time efficient.

Types of Stemmers in NLTK

Snowball Stemmer

The Snowball stemmer is way more aggressive than Porter Stemmer and is also referred to as Porter2 Stemmer. Because of the improvements added when compared to the Porter Stemmer, the Snowball stemmer is having greater computational speed.

- * multi-lingual as it can handle non-English words.

Types of Stemmers in NLTK

Lancaster Stemmer

The Lancaster stemmers are more aggressive and dynamic compared to the other two stemmers. The stemmer is really faster, but the algorithm is really confusing when dealing with small words. But they are not as efficient as Snowball Stemmers. The Lancaster stemmers save the rules externally and basically uses an iterative algorithm.

Types of Stemmers in NLTK

Regex Stemmer

The Regex Stemmer, or Regular Expression Stemmer, is a stemming algorithm that utilizes regular expressions to identify and remove suffixes from words. It allows users to define custom rules for stemming by specifying patterns to match and remove.

Stemming

Stemming:

```
# import these modules
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
ps = PorterStemmer()
# choose some words to be stemmed
words = ["program", "programs",
"programmer", "programming",
"programmers"]
for w in words:
    print(w, " : ", ps.stem(w))
```

O/P:

program : program

programs : program

programmer : programm

programming : program

programmers : programm

Stemming

Stemming:

```
# importing modules
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

ps = PorterStemmer()

sentence = "Programmers program with
programming languages"
words = word_tokenize(sentence)
for w in words:
    print(w, " : ", ps.stem(w))
```

O/P:

Programmers : programm
program : program
with : with
programming : program
languages : languag

Lemmatization

Lemmatization:

- ❖ Lemmatization is the process of grouping together the different inflected forms of a word so they can be analyzed as a single item.
- ❖ Lemmatization is similar to stemming but it brings context to the words.
- ❖ So it links words with similar meanings to one word.

❖ Example:

- ❖ -> rocks : rock
- ❖ -> corpora : corpus
- ❖ -> better : good

Applications of lemmatization are:

- ❖ Used in comprehensive retrieval systems like search engines.
- ❖ Used in compact indexing

Lemmatization

Lemmatization:

likes	→	like
better	→	good
worse	→	bad

Word	Stemming	Lemmatization
information	inform	information
informative	inform	informative
computers	comput	computer
feet	feet	foot

Lemmatization

Techniques:

Rule Based Lemmatization

❖ It involves the application of predefined rules to derive the base or root form of a word. Unlike machine learning-based approaches, which learn from data, rule-based lemmatization relies on linguistic rules and patterns.

- **Rule:** For regular verbs ending in “-ed,” remove the “-ed” suffix.

Example:

Word: “walked”

Rule Application: Remove “-ed”

Result: “walk

Lemmatization

Techniques:

Dictionary-Based Lemmatization

❖ It relies on predefined dictionaries or lookup tables to map words to their corresponding base forms or lemmas. Each word is matched against the dictionary entries to find its lemma. This method is effective for languages with well-defined rules.

Example:

'running' -> 'run'

'better' -> 'good'

'went' -> 'go'

Lemmatization

Techniques:

Machine Learning -Based Lemmatization

❖ Unlike rule-based or dictionary-based approaches, machine learning models, such as neural networks or statistical models, are trained on large text datasets to generalize patterns in language.

❖ Example:

'running' -> 'run'

'better' -> 'good'

'went' -> 'go'

Lemmatization

Lemmatization:

```
# import these modules  
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
```

```
print("rocks :", lemmatizer.lemmatize("rocks"))  
print("corpora :", lemmatizer.lemmatize("corpora"))
```

```
# a denotes adjective in "pos"
```

```
print("better :", lemmatizer.lemmatize("better", pos="a"))
```

O/P:

rocks : rock

corpora : corpus

better : good

Lemmatization

Advantages:

Improves text analysis accuracy

Reduces data size

Better search results

Disadvantages:

Time-consuming

Not suitable for real-time applications

May lead to ambiguity

O/P:

rocks : rock

corpora : corpus

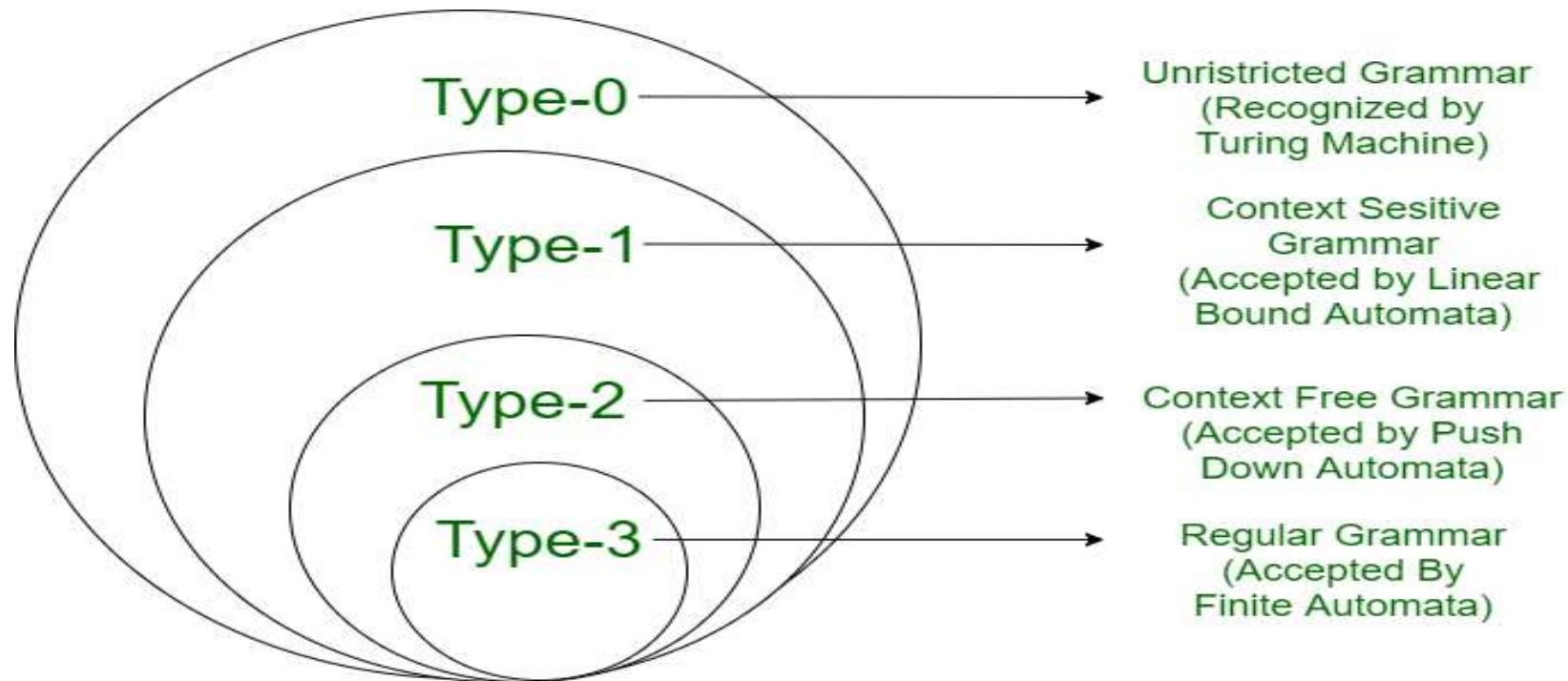
better : good

STEMMING vs LEMMATIZATION

	<u>Stemming</u>	<u>Lemmatization</u>
1	Stemming is faster because it chops words without knowing the context of the word in given sentences.	Lemmatization is slower as compared to stemming but it knows the context of the word before proceeding.
2	It is a rule-based approach.	It is a dictionary-based approach.
3	Accuracy is less.	Accuracy is more as compared to Stemming.
4	When we convert any word into root-form then stemming may create a word with no meaning.	Lemmatization always gives the dictionary meaning while converting words into root-form.
5	It is used when the meaning of the word is not important. Example: Spam Detection	It should be used when it is important to retain the meaning of the word. Example: Question Answer
6	For Example: "Studies" => "Studi"	For Example: "Studies" => "Study"

Grammar

❖ rules of a language governing the sounds, words, sentences, and other elements, as well as their combination and interpretation.



Context Free Grammar

Context free grammar is a formal grammar which is used to generate all possible strings in a given formal language.

Context free grammar G can be defined by four tuples as:

$G = (V, T, P, S)$

Where

V describes a finite set of non-terminal symbols or variables

T describes a finite set of terminal symbols.

P describes a set of production rules

S is the start symbol.

Context Free Grammar

Context free grammar is a formal grammar which is used to generate all possible strings in a given formal language.

Context free grammar G can be defined by four tuples as:

$G = (V, T, P, S)$

Where

V describes a finite set of non-terminal symbols or variables

T describes a finite set of terminal symbols.

P describes a set of production rules

S is the start symbol.

Example

The grammar $(\{A\}, \{a, b, c\}, P, A)$, $P : A \rightarrow aA, A \rightarrow abc$.

The grammar $(\{S\}, \{a, b\}, P, S)$, $P : S \rightarrow aSa, S \rightarrow bSb, S \rightarrow \epsilon$

The grammar $(\{S, F\}, \{0, 1\}, P, S)$, $P : S \rightarrow 00S \mid 11F, F \rightarrow 00F \mid \epsilon$

Context Free Grammar

Generation of Derivation Tree

A derivation tree or parse tree is an ordered rooted tree that graphically represents the semantic information a string derived from a context-free grammar.

Representation Technique

Root vertex – Must be labeled by the start symbol.

Vertex – Labeled by a non-terminal symbol.

Leaves – Labeled by a terminal symbol or ϵ .

Leftmost derivation – A leftmost derivation is obtained by applying production to the leftmost variable in each step.

Rightmost derivation – A rightmost derivation is obtained by applying production to the rightmost variable in each step.

Context Free Grammar

Generation of Derivation Tree or parse tree

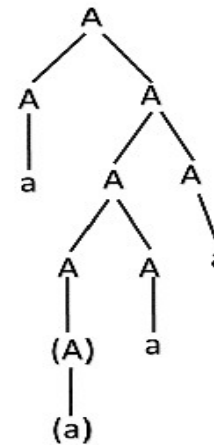
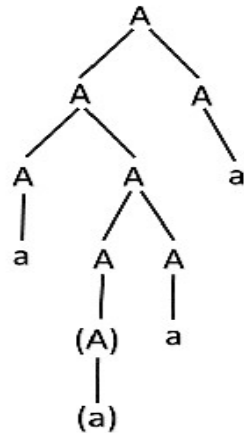
$A \rightarrow AA$

$A \rightarrow (A)$

$A \rightarrow a$

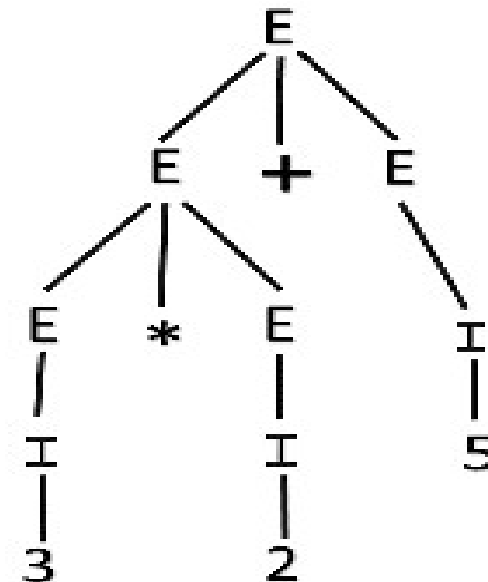
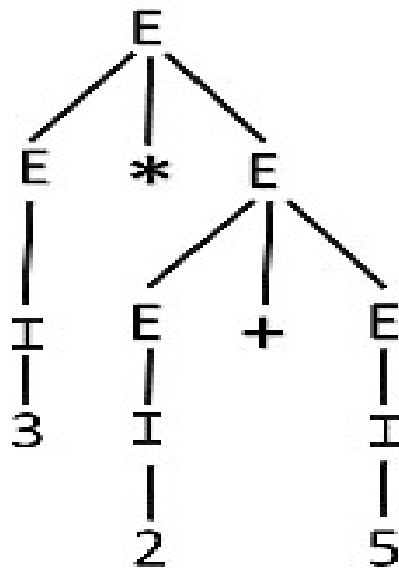
Solution:

For the string "a(a)aa" the above grammar can generate two parse trees:



Context Free Grammar

A CFG is ambiguous if one or more terminal strings have multiple leftmost derivations from the start symbol.



Parts of Speech (POS) Tagging

❖ part-of-speech tagging (POS tagging or PoS tagging or POST), also called grammatical tagging or word-category disambiguation.

❖ Input: I am going.

❖ Output:

with stop word

[('I', 'PRP'), ('am', 'VBP'), ('going', 'VBG'), ('.', '.')]]

Without stop word

[('I', 'PRP'), ('going', 'VBG'), ('.', '.')]]

Where

PRP- Pronoun Personal

VBP-Verb non-3rd personal singular present

VBG-Verb Gerund/ Present Participle

Parts of Speech (POS) Tagging

Sl No.	TAG	DESCRIPTION	EXAMPLE(S)
1	CC	Coordinating Conjunction	<i>and, or</i>
2	CD	Cardinal Number	<i>five, one, 2</i>
3	DT	Determiner	<i>a, the</i>
4	EX	Existential <i>there</i>	<i>there were two cars</i>
5	FW	Foreign Word	<i>d'hoevre, mais</i>
6	IN	Preposition/ Subordinating Conjunction	<i>of, in, on, that</i>
7	JJ	Adjective	<i>quick, lazy</i>
8	JJR	Adjective, comparative	<i>quicker, lazier</i>

Parts of Speech (POS) Tagging

9	JJS	Adjective, superlative	<i>quickest, laziest</i>
10	LS	List item marker	<i>2)</i>
11	MD	Verb, modal	<i>could, should</i>
12	NN	Noun, singular or mass	<i>fox, dog</i>
13	NNS	Noun, plural	<i>foxes, dogs</i>
14	NNP	Noun, proper singular	<i>John, Alice</i>
15	NNPS	Noun, proper plural	<i>Vikings, Indians, Germans</i>
16	PDT	Predeterminer	<i>both the cats</i>
17	POS	Possessive ending	<i>boss's</i>
18	PRP	Pronoun, personal	<i>me, you</i>

Parts of Speech (POS) Tagging

19	PRP\$	Pronoun, possessive	<i>our, my, your</i>
20	RB	Adverb	<i>naturally, extremely, hardly</i>
21	RBR	Adverb, comparative	<i>better</i>
22	RBS	Adverb, superlative	<i>best</i>
23	RP	Adverb, particle	<i>about, up</i>
24	SYM	Symbol	<i>%, \$</i>
25	TO	Infinitival to	<i>how to, what to do</i>
26	UH	Interjection	<i>oh, gosh, wow</i>

Parts of Speech (POS) Tagging

SI No.	TAG	DESCRIPTION	EXAMPLE(S)
27	VB	Verb, base form	<i>run, give</i>
28	VBD	Verb, past tense	<i>ran, gave</i>
29	VBG	Verb, gerund/ present participle	<i>running, giving</i>
30	VBN	Verb, past participle	<i>given</i>
31	VBP	Verb, non-3rd person singular present	<i>I think, I take</i>
32	VBZ	Verb, 3rd person singular present	<i>he thinks, he takes</i>
33	WDT	Wh-determiner	<i>which, whatever</i>
34	WP	Wh-pronoun, personal	<i>who, what</i>
35	WP\$	Wh-pronoun, possessive	<i>whose</i>

Parts of Speech (POS) Tagging

36	WRB	Wh-adverb	<i>where, when</i>
37	NP	Noun Phrase	<i>the brown fox</i>
38	PP	Prepositional Phrase	<i>in between, over the dog</i>
39	VP	Verb Phrase	<i>was jumping</i>
40	ADJP	Adjective Phrase	<i>warm and snug</i>
41	ADVP	Adverb Phrase	<i>also</i>
42	SBAR	Subordinating Conjunction	<i>whether or not</i>
43	PRT	Particle	<i>up</i>
44	INTJ	Interjection	<i>hello</i>
45	PNP	Prepositional Noun Phrase	<i>over the dog, as of today</i>
46	-SBJ	Sentence Subject	<i>the fox jumped over the dog</i>
47	-OBJ	Sentence Object	<i>the fox jumped over the dog</i>

Parts of Speech (POS) Tagging



Learning from Experience, Sharing with Passion

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = " Everything is all about money."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
    # Word tokenizers is used to find the words
    # and punctuation in a string
    wordsList = nltk.word_tokenize(i)
# removing stop words from wordList
wordsList = [w for w in wordsList if not w in stop_words]
# Using a Tagger. Which is part-of-speech
# tagger or POS-tagger.
tagged = nltk.pos_tag(wordsList)
print(tagged)
```

O/P:

```
[('Everything', 'VBG'),
('money', 'NN'),
('.', '.')]

```

Parts of Speech (POS) Tagging

with stop words

```
import nltk
# download required nltk packages
# required for tokenization
nltk.download('punkt')
# required for parts of speech tagging
nltk.download('averaged_perceptron_tagger')

# input text
sentence = """"Today morning, I am Very Happy.""""

# tokene into words
tokens = nltk.word_tokenize(sentence)

# parts of speech tagging
tagged = nltk.pos_tag(tokens)

# print tagged tokens
print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','),
('I', 'PRP'),
('am', 'VBP'),
('Very', 'RB'),
('Happy', 'JJ'), (',', ',')]
```

Parts of Speech (POS) Tagging

```
# Without stop words
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = "Today morning, I am Very Happy."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
# Word tokenizers is used to find the words
# and punctuation in a string
    wordsList = nltk.word_tokenize(i)
    # removing stop words from wordList
    wordsList = [w for w in wordsList if not w in stop_words]
    # Using a Tagger. Which is part-of-speech
    # tagger or POS-tagger.
    tagged = nltk.pos_tag(wordsList)
    print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','), ('I', 'PRP'),
('Very', 'RB'),
('Happy', 'JJ'), (':', ':')]
```